

Spring Security en Spring MVC

Sesión 2: Autorización, Roles, Authorities y
Persistencia en BD

Computación en Internet II — Universidad Icesi

Ruta de Aprendizaje — Sesión 2



01. Autorización a Fondo (AuthZ)

Modelo de control de acceso granular y SecurityContextHolder.



02. Roles vs Authorities

GrantedAuthority, prefijo ROLE_ y reglas hasRole vs hasAuthority.



03. Motor de Autenticación y BD

DaoAuthenticationProvider, UserDetailsService, BCrypt y RBAC.



04. Configuración, @PreAuthorize y Vistas

SecurityFilterChain, seguridad de métodos (@PreAuthorize) y Thymeleaf condicional.

01. Autorización (AuthZ) y Modelo de Seguridad

Control de acceso granular, principio de menor privilegio y contexto en memoria.



Control de Acceso



Menor Privilegio



Filtro AuthZ

Autorización: Control de Acceso Granular



De la Identidad a los Privilegios

En la Sesión 1 confirmamos *quién es el usuario* (Autenticación). Ahora debemos decidir: **¿A qué acciones y recursos tiene permiso?**



Principio de Menor Privilegio (PoLP)

Cada usuario debe contar únicamente con los permisos mínimos indispensables para realizar sus funciones operativas.



Evaluación Centralizada en Filtros

La autorización se resuelve antes de que el controlador comience a procesar la lógica de negocio.

El Modelo de Objetos de Seguridad

La Jerarquía del Contexto



SecurityContextHolder

Almacena el contexto de seguridad mediante `ThreadLocal` (aislado para cada hilo de petición HTTP).



SecurityContext

Envoltorio que contiene el objeto `Authentication` actualmente validado.



Authentication

Representa al usuario logueado con sus datos de identidad y lista de permisos concedidos.

Las 4 Dimensiones de Authentication

1. Principal (`getPrincipal`)

Identidad del usuario: Instancia de `UserDetails` / `CustomUserDetails`.

2. Credentials (`getCredentials`)

Contraseña plana/token. Se limpia con `eraseCredentials()` tras validar.

3. Authorities (`getAuthorities`)

Lista de `GrantedAuthority`: Roles (`ROLE_*`) y permisos atómicos.

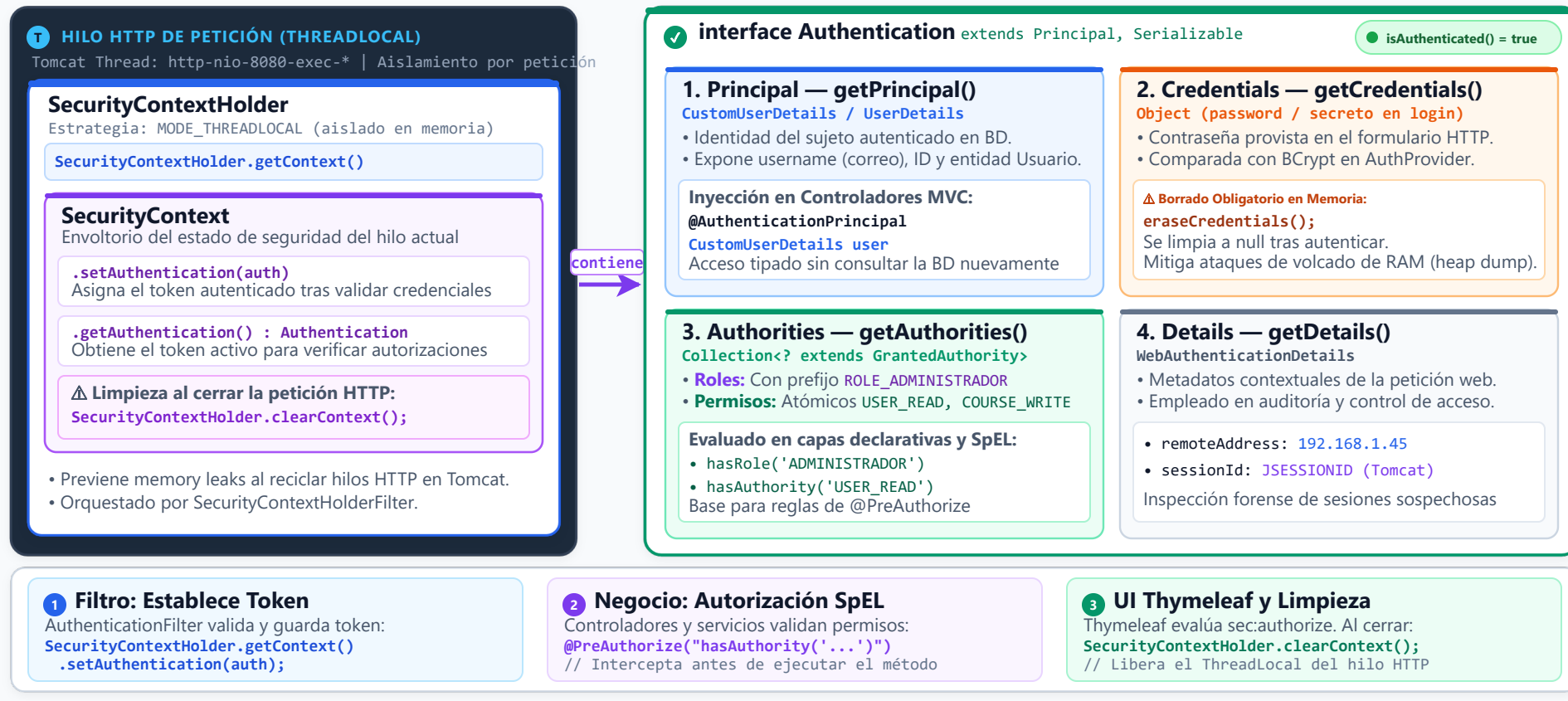
4. Details (`getDetails`)

Metadatos web de la petición HTTP (IP remota y Session ID de Tomcat).

Anatomía de Authentication e Interacción con el Contexto

Anatomía del Objeto Authentication e Interacción con el Contexto

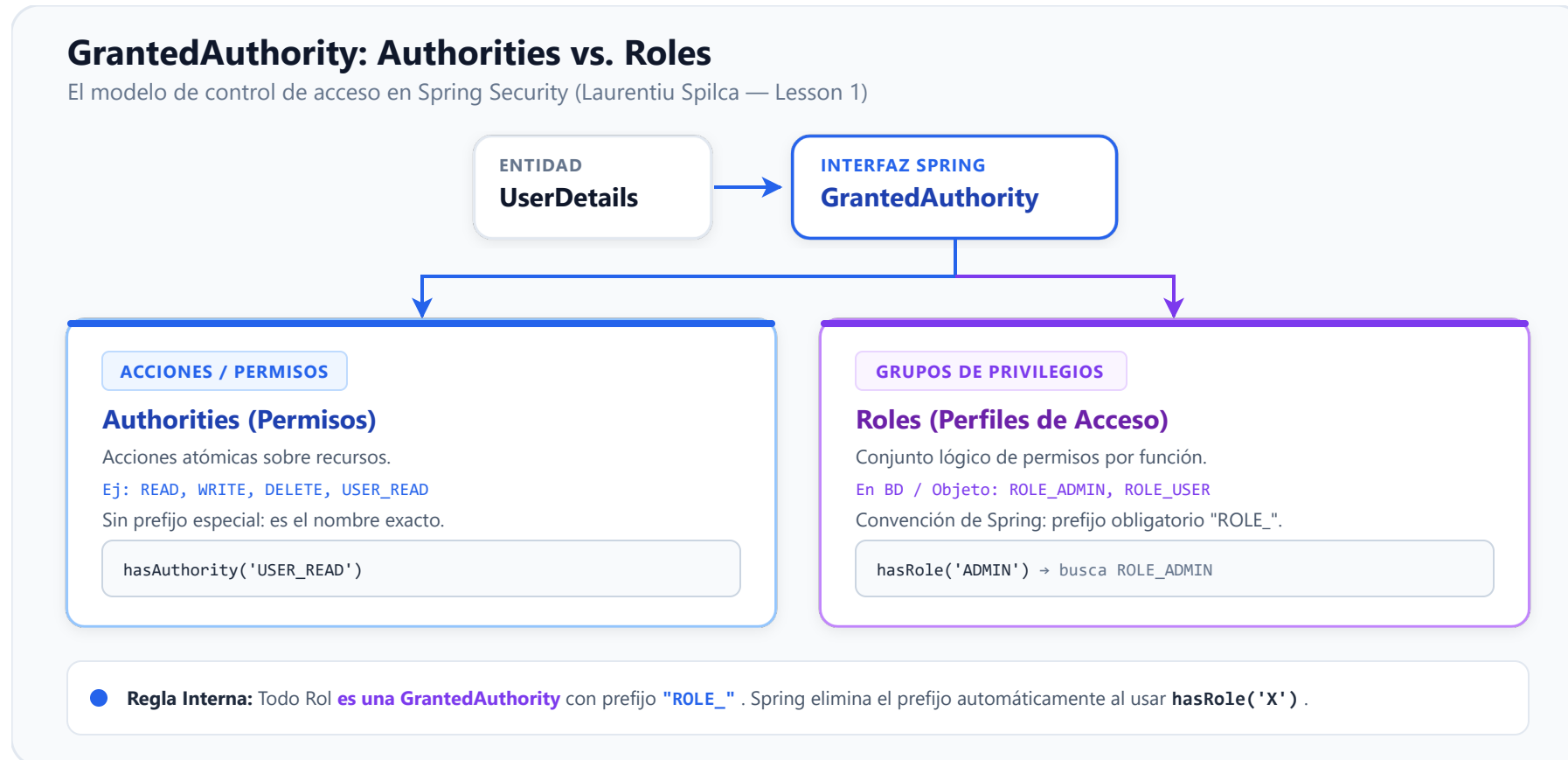
Jerarquía ThreadLocal: SecurityContextHolder → SecurityContext → Authentication (Principal, Credentials, Authorities, Details)



02. Roles vs Authorities: Diseño Limpio

GrantedAuthority, prefijo ROLE_, granularidad y reglas hasRole vs hasAuthority.

GrantedAuthority: Authorities vs. Roles



Reglas de Sintaxis: `hasRole()` vs `hasAuthority()`

HASROLE("ADMIN")

`hasRole("ADMIN")`

Spring Security agrega automáticamente el prefijo `ROLE_` internamente.

Busca que el usuario posea la authority `ROLE_ADMIN`.

HASAUTHORITY("ROLE_ADMIN")

`hasAuthority("ROLE_ADMIN")`

Busca coincidencia exacta de texto. No agrega ningún prefijo.

Exige especificar el nombre completo de la autoridad.

HASAUTHORITY("USER_CREATE")

`hasAuthority("USER_CREATE")`

Se utiliza para validar permisos granulares. Permite proteger endpoints basados en acciones específicas sin depender del rol global.

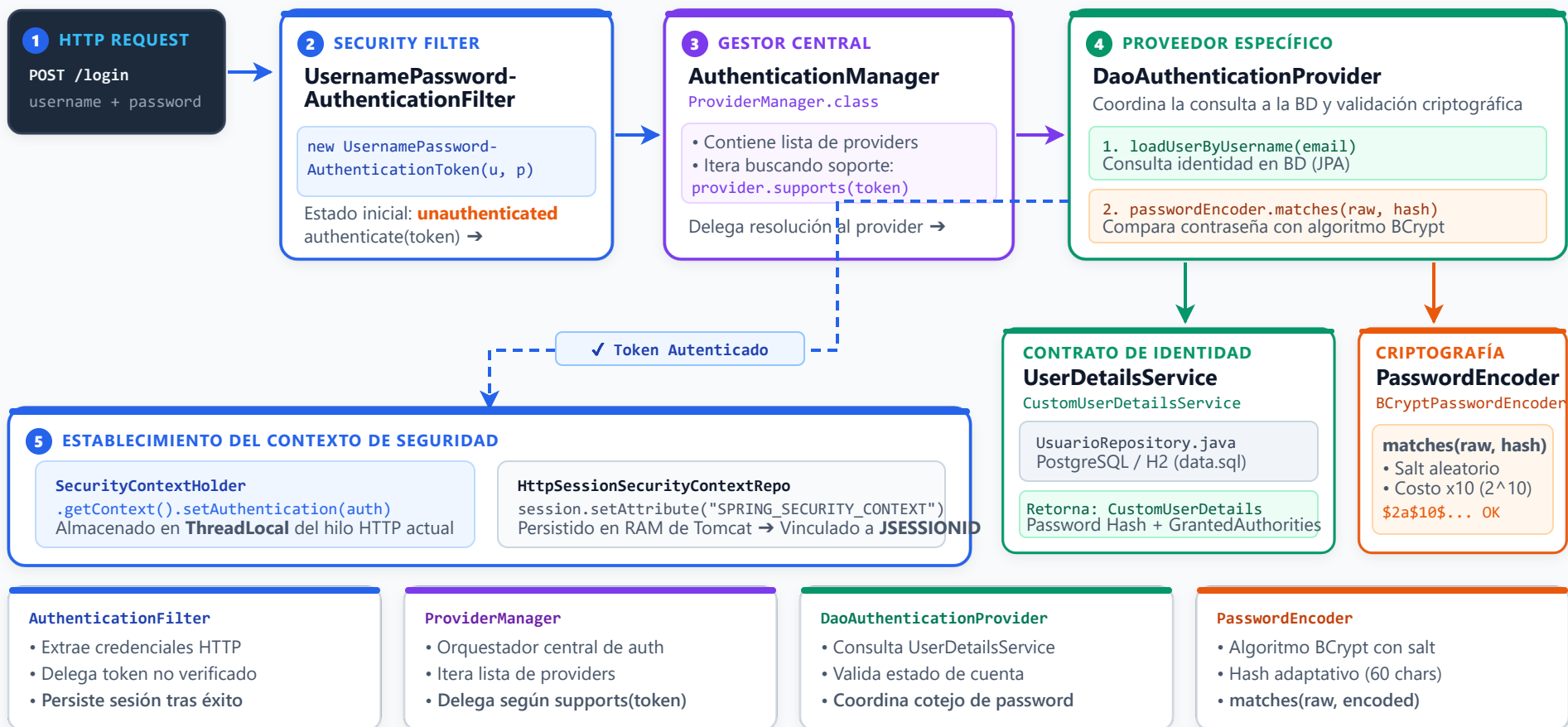
03. Motor de Autenticación y Persistencia JPA

DaoAuthenticationProvider, UserDetailsService, BCrypt y modelo relacional RBAC.

Arquitectura del Motor de Autenticación

Arquitectura del Motor de Autenticación

Flujo de resolución: AuthenticationFilter → ProviderManager → DaoAuthenticationProvider → BD & BCrypt



Contratos: UserDetailsService y UserDetails

Desacoplamiento en Spring Security

Spring Security no sabe cómo está construida tu tabla de usuarios ni qué campos tiene tu entidad JPA.



UserDetailsService

Interfaz funcional con un único método: `loadUserByUsername(String username)`.



UserDetails

Contrato que expone los datos requeridos por Spring: `getUsername()`, `getPassword()` y `getAuthorities()`.

```
@Service
public class CustomUserDetailsService
    implements UserDetailsService {

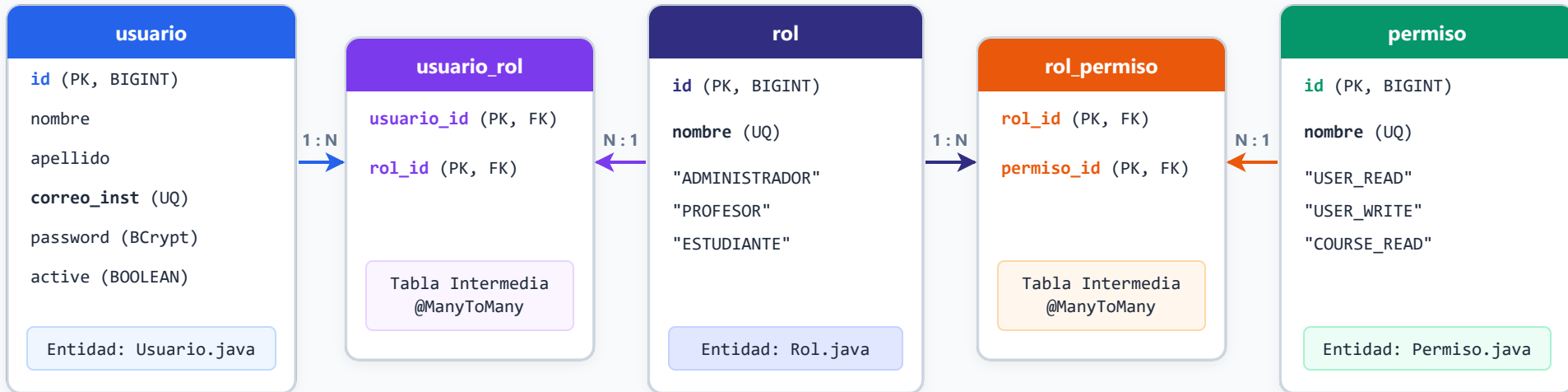
    @Autowired
    private UsuarioRepository usuarioRepository;

    @Override
    public UserDetails loadUserByUsername(String email)
        throws UsernameNotFoundException {
        Usuario usuario = usuarioRepository
            .findByCorreoInstitucional(email)
            .orElseThrow(() ->
                new UsernameNotFoundException("No existe:" + email));
        return new CustomUserDetails(usuario);
    }
}
```

Modelado Relacional RBAC en Base de Datos

Modelo Relacional de Control de Acceso (RBAC en BD)

Estructura de tablas en PostgreSQL/H2 (schema.sql / data.sql) mapeadas a JPA en el proyecto springboot



Mapeo a Spring Security en CustomUserDetails (GrantedAuthority):

- Roles de Alto Nivel:** Se mapean con prefijo "ROLE_" + rol.getNombre() (ej. ROLE_ADMINISTRADOR) para evaluar con hasRole('ADMINISTRADOR').
- Permisos Atómicos:** Se mapean directamente con permiso.getNombre() (ej. USER_READ, COURSE_WRITE) para evaluar con hasAuthority('USER_READ').

Adaptadores: CustomUserDetails y Authority

```
public class SecurityAuthority
    implements GrantedAuthority {
    private final Permiso permiso;

    public SecurityAuthority(Permiso permiso) {
        this.permiso = permiso;
    }

    @Override
    public String getAuthority() {
        return permiso.getNombre();
    }
}
```

```
@AllArgsConstructor
public class CustomUserDetails implements UserDetails {
    private final Usuario usuario;

    @Override
    public Collection<? extends GrantedAuthority> getAuthorities() {
        Set<GrantedAuthority> auths = new HashSet<>();
        for (Rol rol : usuario.getRoles()) {
            auths.add(new SimpleGrantedAuthority("ROLE_" + rol.getNombre()));
            for (Permiso p : rol.getPermisos()) {
                auths.add(new SimpleGrantedAuthority(p.getNombre()));
            }
        }
        return auths;
    }
}
```

Servicio: CustomUserDetailsService con JPA

Carga Transaccional de Datos

El servicio busca la entidad `Usuario` en BD por correo institucional y garantiza la carga de relaciones ManyToMany dentro de la transacción.



UsuarioRepository

Método derivado: `findByCorreoInstitucional(String email)`.



@Transactional(readOnly = true)

Mantiene la sesión de Hibernate abierta para inicializar roles y permisos sin `LazyInitializationException`.

```
@Service
@RequiredArgsConstructor
public class CustomUserDetailsService implements UserDetailsService {

    private final UsuarioRepository usuarioRepository;

    @Override
    @Transactional(readOnly = true)
    public UserDetails loadUserByUsername(String email)
        throws UsernameNotFoundException {

        Usuario u = usuarioRepository.findByCorreoInstitucional(email)
            .orElseThrow(() -> new UsernameNotFoundException(
                "Usuario no encontrado: " + email));

        // Forzar carga de relaciones Lazy en la sesión Hibernate
        u.getRoles().forEach(r -> r.getPermisos().size());
        return new CustomUserDetails(u);
    }
}
```

04. Configuración Declarativa, @PreAuthorize y Vistas

SecurityFilterChain, seguridad de métodos (@PreAuthorize) y renderización condicional con Thymeleaf.

Reglas Declarativas de Autorización de Rutas

Reglas Declarativas en HttpSecurity

En Spring Boot 3+, la configuración se realiza definiendo un Bean de tipo `SecurityFilterChain` mediante expresiones Lambda.



requestMatchers + permitAll()

Rutas públicas para login, recursos estáticos (/css, /js) o registro.



requestMatchers + hasRole()

Rutas protegidas por perfil o privilegio específico.



anyRequest().authenticated()

Cualquier otra ruta exige obligatoriamente sesión activa.

```
@Bean
public SecurityFilterChain filterChain(HttpSecurity http)
    throws Exception {
    return http
        .authorizeHttpRequests(authz -> authz
            .requestMatchers("/css/**", "/js/**", "/images/**").permitAll()
            .requestMatchers("/roles-mvc/**", "/permisos-mvc/**")
                .hasRole("ADMINISTRADOR")
            .requestMatchers("/profesores-mvc/**")
                .hasAnyRole("ADMINISTRADOR", "PROFESOR")
            .requestMatchers("/cursos-mvc/**", "/matriculas-mvc/**").authenticated()
            .anyRequest().authenticated()
        )
        .formLogin(form -> form
            .loginPage("/login")
            .defaultSuccessUrl("/usuarios", true)
            .permitAll()
        )
        .logout(logout -> logout
            .logoutUrl("/logout")
            .logoutSuccessUrl("/login?logout")
            .permitAll()
        )
        .build();
}
```



Vista Custom



Error 401/Cred



403 Denied

Integración del Login y Manejo de 403 Forbidden



Conexión con BD y UserDetails

El formulario básico (username, password) creado en Sesión 1 delega transparente en DaoAuthenticationProvider y BD.



failureUrl("/login?error")

Redirecciona con parámetro si las credenciales fallan, activando el mensaje de alerta condicional en Thymeleaf.



accessDeniedPage("/access-denied")

Cuando un usuario autenticado (ej. ESTUDIANTE) intenta violar permisos, se redirige a la vista amigable 403.

```
//En SecurityConfig.java
http
    .formLogin(form -> form
        .loginPage("/login")
        .defaultSuccessUrl("/usuarios", true)
        .failureUrl("/login?error")
        .permitAll()
    )
    .exceptionHandling(ex -> ex
        .accessDeniedPage("/access-denied")
    );
```

Seguridad a Nivel de Métodos: @PreAuthorize

Control Granular con Spring AOP y SpEL

Proteger rutas perimetrales con `requestMatchers` no basta. `@PreAuthorize` valida permisos atómicos directamente en la invocación de métodos en Controladores y Servicios.



@EnableMethodSecurity

Anotación obligatoria en SecurityConfig (reemplaza al viejo `@EnableGlobalMethodSecurity`).



Expresiones SpEL Flexibles

`hasAuthority('USER_DELETE')` o `hasAnyRole('ADMINISTRADOR', 'PROFESOR')`.

```
//1. Habilitar en SecurityConfig.java
@Configuration
@EnableWebSecurity
@EnableMethodSecurity // <-- Obligatorio
public class SecurityConfig { ... }

//2. Proteger Métodos en UsuarioController.java
@Controller
@RequestMapping("/usuarios")
public class UsuarioController {

    // Solo quien tenga el permiso atómico
    @PreAuthorize("hasAuthority('USER_DELETE')")
    @PostMapping("/eliminar/{id}")
    public String eliminar(@PathVariable Long id) {
        usuarioService.eliminar(id);
        return "redirect:/usuarios";
    }

    // Administradores o Docentes
    @PreAuthorize("hasAnyRole('ADMINISTRADOR', 'PROFESOR')")
    @GetMapping("/detalle/{id}")
    public String verDetalle(...) { ... }
}
```

Defensa en Profundidad: Las 3 Capas de Seguridad

CAPA 1: UI (THYMELEAF)

sec:authorize

Oculta o muestra botones y enlaces según el perfil.

Beneficio: Experiencia de usuario limpia (UX).

⚠️ No sustituye la seguridad del backend.

CAPA 2: WEB HTTP (FILTER)

authorizeHttpRequests

Intercepta peticiones en `SecurityFilterChain` por prefijos URL.

Beneficio: Cortafuegos perimetral para rutas completas (/roles-mvc/**).

CAPA 3: LÓGICA JAVA (MÉTODO)

@PreAuthorize

Interceptor AOP que valida el `SecurityContext` antes de ejecutar el método Java.

Beneficio: Protección granular imposible de evadir aunque se manipule la URL.

Gestión Segura de Logout

1. ENDPOINT DE SALIDA

logoutUrl("/logout")

Ruta interceptada por `LogoutFilter` (por defecto POST para proteger contra ataques CSRF).

2. INVALIDACIÓN EN SERVIDOR

Invalidación de Sesión

Destruye la `HttpSession` en la memoria RAM del servidor y limpia el `SecurityContextHolder`.

3. ELIMINACIÓN DE COOKIE

Limpieza de Cookies

Envía un header para eliminar la cookie `JSESSIONID` en el navegador y redirige a `/login?logout`.

Control de Vistas con Thymeleaf Extras

Renderización Condicional por Roles y Authorities

Con `thymeleaf-extras-springsecurity6`, la interfaz se adapta dinámicamente tanto a roles de alto nivel como a permisos atómicos.



`sec:authorize="hasRole('ADMINISTRADOR')"`

Muestra módulos completos (ej. Gestión de Roles) solo a perfiles administradores.



`sec:authorize="hasAuthority('USER_DELETE')"`

Condiciona botones o acciones destructivas a la posesión del permiso atómico.



`sec:authentication="name"`

Despliega el correo institucional del usuario autenticado en el encabezado.

```
<!-- templates/fragments/layout.html (Fragmento) -->
<header th:fragment="mainNavbar">
  <nav>
    <a th:href="@{/usuarios}">Usuarios</a>
    <a th:href="@{/roles-mvc}"
    sec:authorize="hasRole('ADMINISTRADOR')">Roles</a>
    <span>Usuario: <strong sec:authentication="name"></strong></span>
    <form th:action="@{/logout}" method="post">
      <button type="submit">Salir</button>
    </form>
  </nav>
</header>

<!-- En templates/usuarios/lista.html -->
<td>
  <form th:action="@{/usuarios/eliminar/{id}(id=${u.id})}"
  method="post" sec:authorize="hasAuthority('USER_DELETE')">
    <button type="submit">Eliminar</button>
  </form>
</td>
```

Errores Frecuentes y Buenas Prácticas

LAZY LOADING EN JPA

LazyInitializationException

Ocurre al acceder a `usuario.getRoles()` fuera de sesión Hibernate. *Solución: Usar `@Transactional(readOnly = true)` e inicializar colecciones.*

PREFIJO DE ROLES

Confusión con Prefijo ROLE_

`hasRole('ADMIN')` busca `ROLE_ADMIN`.
No ingresar `hasRole('ROLE_ADMIN')` porque buscaría `ROLE_ROLE_ADMIN`.

OLVIDO DE @ENABLEMETHODSECURITY

@PreAuthorize Ignorado

Si no se anota la configuración con `@EnableMethodSecurity`, los métodos anotados se ejecutarán sin ningún control.

TOKENS CSRF EN POST

403 en Logout / Delete

Formularios HTML puros sin Thymeleaf fallarán con 403 Forbidden. Usar siempre `th:action` para inyectar `_csrf`.

Live Demo 3: Prueba Integral de Acceso y @PreAuthorize

Actividad en Vivo: Validación RBAC y Seguridad de Métodos



1. Login Administrador (admin@icesi.edu.co / admin123)

Verificar acceso total a /roles-mvc y presencia del botón Eliminar.



2. Login Docente (drincon@icesi.edu.co / profesor123)

Acceder a /profesores-mvc pero recibir bloqueo 403 en /roles-mvc.



3. Ocultamiento en UI (apaez@icesi.edu.co / estudiante123)

Comprobar que en la UI desaparecen los menús y botones no autorizados.



4. Forzar Invocación con @PreAuthorize

Intentar POST manual a método protegido: el interceptor AOP bloquea con 403 Forbidden.

// Comprobación de Seguridad AOP

```
org.springframework.security.access.
```

```
AccessDeniedException:
```

```
Access Denied (PreAuthorize check failed  
for authority: USER_DELETE)
```

```
→ Redirigido a /access-denied (403)
```

Laboratorio Práctico

Implementando Spring Security en el Proyecto
MVC